

Single Page Application, Framework JS

Summary

- Single Page Applications
- Framework JS
 - Introduzione
 - Confronto
- Introduzione a VUE

Single Page Application

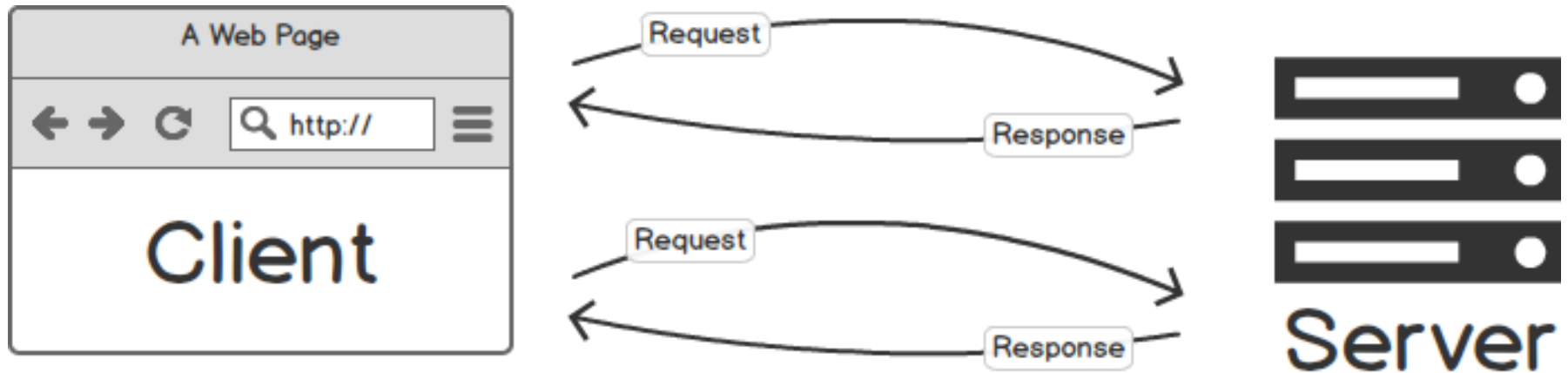
Siti web tradizionali - Frontend

- Riepilogo puntate precedenti ...



→ They make our site more

Server-based Application



Problemi – Lato Client

- La pagina viene caricata completamente ogni volta che viene eseguita una richiesta al server.
- L'utente deve aspettare che tutti i file siano scaricati (HTML, CSS, JS, immagini).
- Il contenuto scaricato deve essere renderizzato.
- Nessuna business logic client side
- Nessun modello MVC
- Senza rete l'applicazione non funziona (funzionamento solo **online**)

Problemi – Lato Server

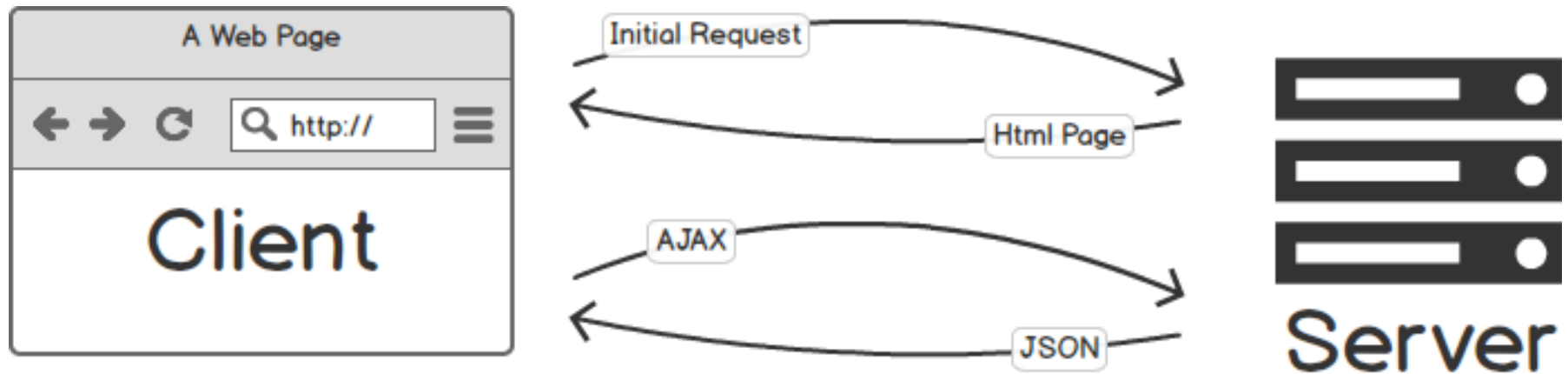
- Ad ogni richiesta la pagina deve essere generata dall'inizio.
- Banda consumata per inviare sempre gli stessi file.
- Tutte le computazioni sono eseguite lato server.
- Un cambio di tecnologia implica la riscrittura completa del codice che gestisce il rendering della pagina.

Single Page Application

Una possibile soluzione ...

- «A Single Page Application is a web app in which the majority of interactions are handled on the client without the need to reach a server with a goal of providing a more fluid user experience»

Single Page Application - Interazione



Vantaggi

- Interazione App-like.
- Bottoni Indietro/Avanti funzionano.
- Possibilità di funzionamento anche ***offline***.
- Transizioni fluide tra i diversi stati delle pagine.
- CSS e JS inviati solamente nella richiesta iniziale.
- I cambiamenti alla pagina avvengo via Javascript, usando Template e la manipolazione del DOM.

JQuery Ninja?

- Vari problemi:
 - Stato dell'applicazione risiede nel DOM
 - Lo stato guida il comportamento
 - Si accoppia logica e presentazione
 - Non è pensato per gestire la logica di un'applicazione

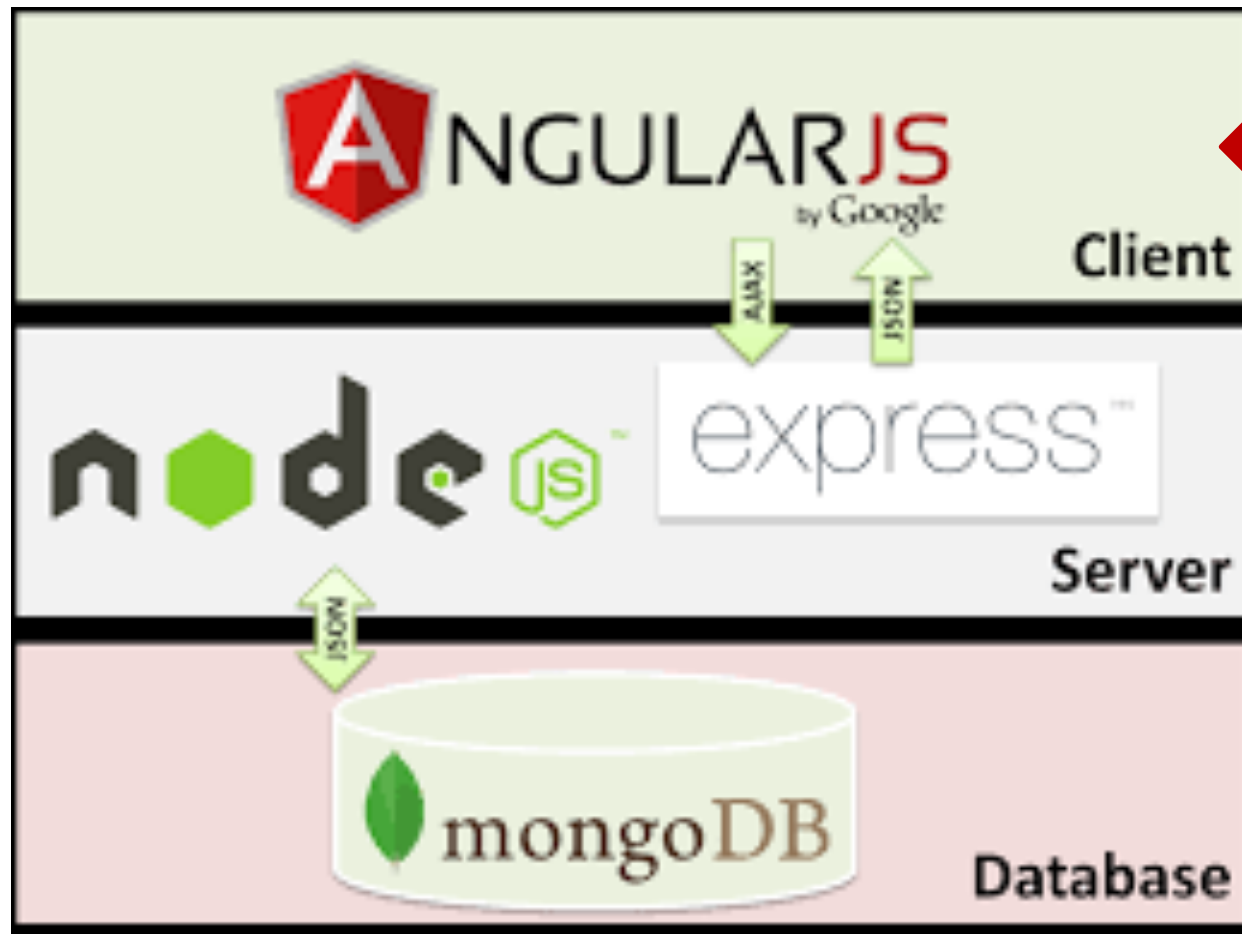


Storia

- Nascono diverse tecnologie (librerie/framework) per le SPA
- Conferenza Throne of JS a Toronto (2012)
 - Gli sviluppatori dei maggiori framework JS si riuniscono per discutere come affrontare il bisogno delle nuove architetture SPA

Framework JS

AngularJS



Angular(JS)

- In realtà, dalla sua versione 2, si chiama Angular (senza il js)
- È un framework javascript lato client, per questo motivo se ne può utilizzare uno diverso
- I più diffusi sono
 - Angular
 - REACT
 - VUE

AngularJS

- Release 1.0 nel 2012.
- Divenne subito molto popolare
- Nel picco della sua popolarità (2014) il team di sviluppo annunciò una nuova versione *Angular 2 incompatibile con la versione precedente* (in pratica una completa riscrittura del framework)
- Nel settembre 2016 viene rilasciato **Angular 2**, mentre a marzo 2017 viene rilasciata la versione 4. L'ultima versione rilasciata è la **15** (Febbraio 2023)

Altri Framework

- Nel 2013 Facebook rilascia **React** (open source) dopo averlo usato in produzione per almeno due anni
- Nel 2014 viene rilasciato **Vue.js** da Evan You, che aveva lavorato precedentemente nel team di sviluppo di AngularJS

Quale usare?

- ... Dipende!
- Vediamo le maggiori differenze in termini di:
 - Storia e longevità
 - Popolarità ed ecosistema
 - Supporto aziendale
 - Esperienza iniziale e curva di apprendimento
 - Capacità richieste
 - Completezza e performance

Storia e longevità

- **Angular:**

- prima release nel 2010, con il nome di AngularJS. La seconda versione è stata completamente riscritta ed è stata rilasciata nel 2016
- realizzazione di applicazioni di nuova generazione, come le applicazioni a pagina singola (SPA)

Storia e longevità

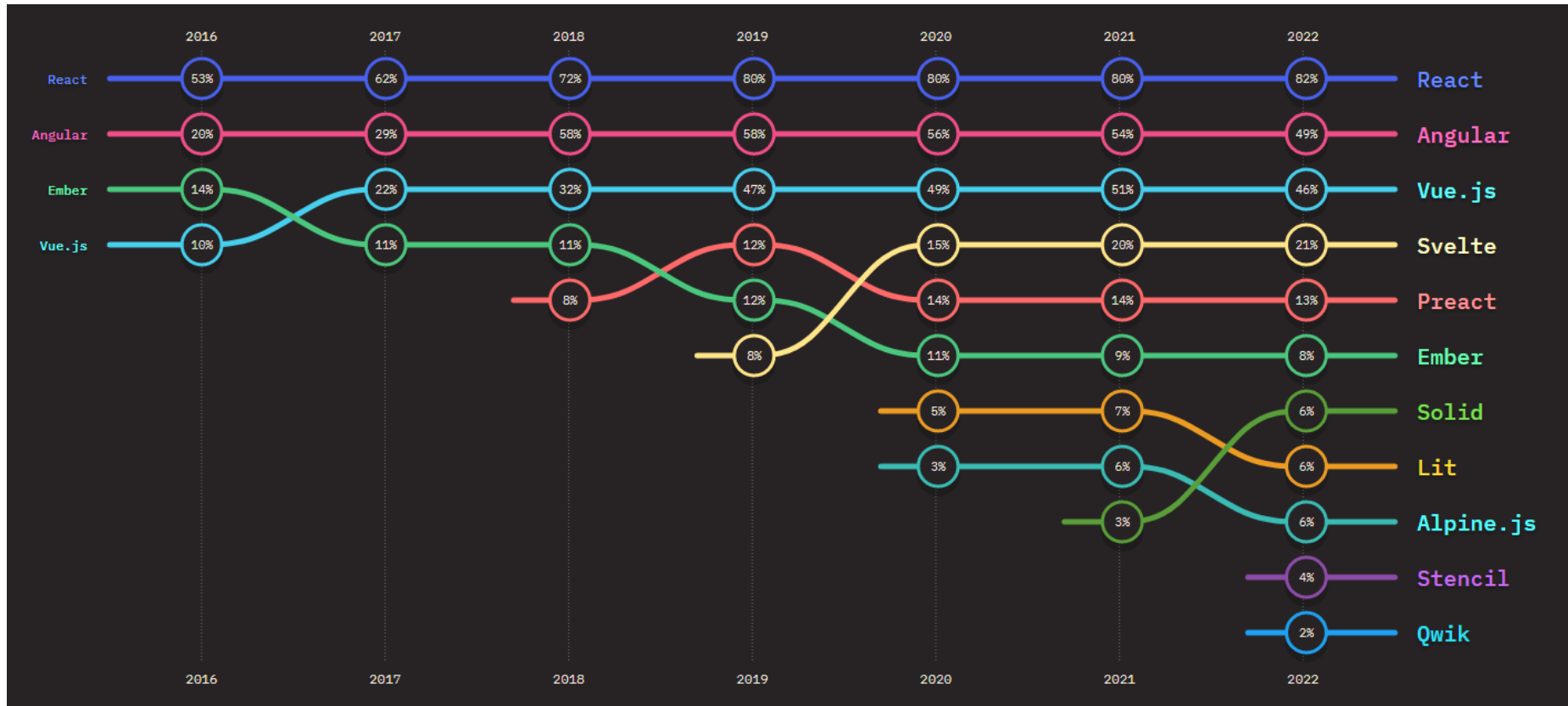
- **REACT**

- talvolta identificato con React.js/ReactJS, rilasciato nel marzo 2013
- creazione di interfacce utente (UI) dedicate ad applicazioni web complesse, in modo semplice, veloce e scalabile

Storia e longevità

- **Vue.js**
 - rilasciato nel febbraio 2014
 - Curiosità: Il creatore del progetto, Evan You, fu uno sviluppatore di Google che utilizzò intensivamente AngularJS per la realizzazione di svariati progetti
 - è stato prodotto per essere un'alternativa "leggera" ad Angular
 - L'autore voleva fornire agli sviluppatori un framework dalla curva di apprendimento bassa, ma che potesse crescere fino a poter gestire applicazioni molto complesse

Popolarità



Ecosistema

Framework	Stelle GitHub	Tagged Questions su StackOverflow
Angular	80.5 K	274.5 K
React	185 K	376 K
Vue.js	195 K	91.5 K

Framework	Numero di pacchetti npm	Download npm (nell'ultima settimana)
Angular	65.660	2.792 K
React	227.918	20.024 K
Vue.js	74.110	3.954 K

Supporto aziendale

Framework	Sponsor
Angular	Google
React	Facebook
Vue.js	Supporto da svariate organizzazioni di piccola/media dimensione, principalmente attraverso Patreon, un tool online dedicato ai creatori di servizi

Esperienza iniziale e curva d'apprendimento

- **Angular:** esperienza iniziale di utilizzo più complessa
- Per lanciare un'applicazione è necessario installare la CLI (command line interface for Angular), ed impostare alcuni parametri:

```
// installazione CLI
npm install -g @angular/cli
// creazione di un nuovo progetto
ng new my-app
// avvio del server ('open' apre automaticamente il
// browser)
cd my-app ng serve --open
```

Esperienza iniziale e curva d'apprendimento

- **Angular:** A questo punto sarà possibile lavorare con i file dei componenti generati da Angular, come *src/app/app.component.ts*

```
export class AppComponent {  
    title = 'Mio titolo';  
}
```

- Per iniziare, occorre seguire il tutorial: **Tour of Heroes**

Esperienza iniziale e curva d'apprendimento

REACT: offre due guide differenti per iniziare a scrivere codice avvalendosi delle sue funzionalità.

- La prima, molto semplice, racchiude lo snippet (con tanto di spiegazione) necessario per generare il classico “Hello World”:

```
ReactDOM.render (  
<h1>Hello, world!</h1>,  
document.getElementById( 'root' )  
) ;
```

Esperienza iniziale e curva d'apprendimento

- **REACT**: assume che lo sviluppatore sia in grado di scrivere applicazioni Javascript conformi agli ultimi standard ECMA (6)
- Per esempio:
 - keywords **let** e **const** per definire variabili
 - la keyword **class** per definire le classi
 - le arrow functions, **=>**

Piccola Parentesi: =>

// (param1, param2, paramN) => expression

// prima

```
var multiplyES5 = function(x, y) {  
    return x * y;  
};
```

// ora

```
const multiplyES6 = (x, y) => { return x * y  
};
```

Esperienza iniziale e curva d'apprendimento

- La seconda guida (step-by-step) mostra tutti i concetti essenziali di **React** ad un livello decisamente più alto, permettendo di creare una prima applicazione completa
- Può essere completata in un'ora di tempo dalla maggior parte degli sviluppatori

Esperienza iniziale e curva d'apprendimento

- **Vue.js:** per installare basta includere un tag `<script>` in un documento HTML

```
<script src="https://cdn.jsdelivr.net/npm/vue"></script>
```

- Nella guida ufficiale sono spiegate le procedure più composite di installazione, che comprendono anche l'uso della CLI `vue-cli` che è però sconsigliata a chi si trova ad utilizzare Vue.js per la prima volta o che ha scarsa dimestichezza con le strutture di Node.js

Esperienza iniziale e curva d'apprendimento

- **Vue.js:** La sintassi è basata su un sistema che permette di produrre il DOM a partire da un apposito template:

```
import {createApp} from 'vue'
Create({
  data() {
    return {count: 0}
  }
}).mount('#app')
<div id="app">
  <button @click="count++">Count is {{count }}</button>
</div>
```

- **Vue** è un framework progressivo: alcune componenti non sono incluse di default (ma ha librerie ufficiali che si occupano di aggiungere le varie funzionalità corredate da un'ottima documentazione)

Esperienza iniziale e curva d'apprendimento

- La curva di apprendimento migliore è quella di **Vue**, seguito da **React**, che richiede un tempo leggermente superiore da dedicare all'apprendimento.
- **Angular** è distanziato (il suo scopo è quello di consentire la realizzazione di applicazioni di nuova generazione anche a livelli di complessità elevati)

Requirements

- **React** richiede necessariamente l'utilizzo del Javascript «moderno» (ECMA script 6+)
- **React** fa uso di **JSX**, una sorta di “estensione” della sintassi Javascript utile a mescolare HTML e JS in un unico codice
- NB: **JSX** non è obbligatorio per utilizzare **React**, MA è quasi impossibile trovare un'applicazione reale che non ne faccia uso

Esempio JSX (React)

```
function formatName(user) {  
  return user.firstName + ' ' + user.lastName;  
}  
  
const user = { firstName: 'Harper', lastName: 'Perez' };  
const element = ( <h1> Hello, { formatName(user)}! </h1> );  
  
ReactDOM.render( element, document.getElementById('root') );
```

Requirements

- **Vue.js** è il framework che risulta più familiare allo sviluppatore web (che proviene da librerie come jQuery)
- Si basa su componenti quali HTML, Javascript (ECMA 5 o 6) e CSS nella loro versione tradizionale, con qualche eccezione, per la costruzione di UI e la stilizzazione
- In modo simile ad Angular, **Vue** utilizza attributi HTML “custom” per gestire compiti quali data-binding e controllo del flusso

Piccoli esempi in VUE

```
<a v-bind:href="url"></a>
```

```
<a :href="url"></a>
```

```
<button v-bind:disabled="someDynamicCondition">
```

```
Button</button>
```

```
<button:disabled="someDynamicCondition">Button</button>
```

Capacità richieste

- **Angular** è stato scritto utilizzando TypeScript
- Per questo motivo, occorre conoscere TypeScript per lavorare con **Angular**
- Uno sviluppatore Javascript “standard” avrà maggiori difficoltà, dato che dovrà prendere confidenza con un linguaggio in cui la tipizzazione è statica, mentre un programmatore che proviene da C# o Java, linguaggi d’eccellenza a tipizzazione statica, si troveranno maggiormente a loro agio

Requirements

Framework	Requisiti
React	JS (ES6+), JSX, CSS-in-JS
Vue.js	HTML, JS (ES5+), CSS
Angular	TypeScript

Completezza

Framework	Strumenti e scopo principale
Angular	All-in-one
React	UI management
Vue	Livello View, componenti aggiuntivi ufficiali

Completezza

- **Angular** è il framework nativamente più completo, con la sua struttura “*all-in-one*” che fornisce UI management, state management, routing, test end-to-end e molto altro
- **React** è improntato principalmente sulla UI, il che significa che per funzionalità avanzate è necessario estendere il suo core tramite strumenti esterni (es. Redux e MobX per la gestione dello stato delle applicazioni web)

Completezza

- **Vue.js** è un framework “progressivo”, che fornisce un core in grado di produrre applicazioni sfruttando la sua logica ed il suo motore di base
- Per le funzionalità aggiuntive è però necessario utilizzare componenti extra
 - Esistono svariati componenti di alta qualità, che coprono tutte le principali caratteristiche richieste da un’applicazione di nuova generazione, sia prodotte da terze parti, sia ufficiali, come Vuex (per lo state management centralizzato)
- Il core di Vue è infatti principalmente basato sul livello delle “View”

Completezza

- Riassumendo ...
- Una soluzione full-featured come **Angular** potrebbe NON essere la scelta migliore nel caso in cui si stia realizzando un'applicazione che non necessiti di tutte le sue funzionalità, oppure una che deve essere realizzata in tempi brevi
- Nel caso di framework più leggeri (e nativamente meno completi) come **Vue** e **React**, abbiamo la possibilità di scegliere tra estensioni ufficiali (caso di **Vue**) oppure tra un ampio ecosistema di librerie aggiuntive di terze parti (caso di **React**)

Fattori di performance

- A causa della sua completezza “all-in-one”, **Angular** risulta essere la libreria più pesante, segue **React** e infine **Vue**
- Risultati di benchmark mostrano una leggera deviazione di performance tra i tre framework, dove **Angular** risulta il più “lento” e **Vue** il più “veloce”

VUE

- Si può apprendere rapidamente
- Vicino alla versione “classica” di HTML/CSS/Javascript
- Si può espandere progressivamente a seconda dell’espansione della complessità dell’applicazione
- Veloce e leggero

Q&A

- Ci sono domande?

Riferimenti

- <https://www.html.it/guide/vue-js-la-guida/>
- <https://vuejs.org/v2/guide/>
- <https://www.html.it/articoli/ecmascript-6-harmony-il-javascript-che-verra/>
- <https://angular.io/tutorial>
- <https://www.html.it/articoli/angular-react-vue-framework-javascript-a-confronto/>
- <https://reactjs.org/docs/introducing-jsx.html>
- <https://www.stefankrause.net/js-frameworks-benchmark6/webdriver-ts-results/table.html>
- <https://www.html.it/pag/69928/direttive-personalizzate-2/>